abcd

aabc

abcd

i/p: aabc
o/p: a # b b

i/p: "aacb"
o/p: "a # c c"

a → ~~x~~ 2
b → 1
c → 1

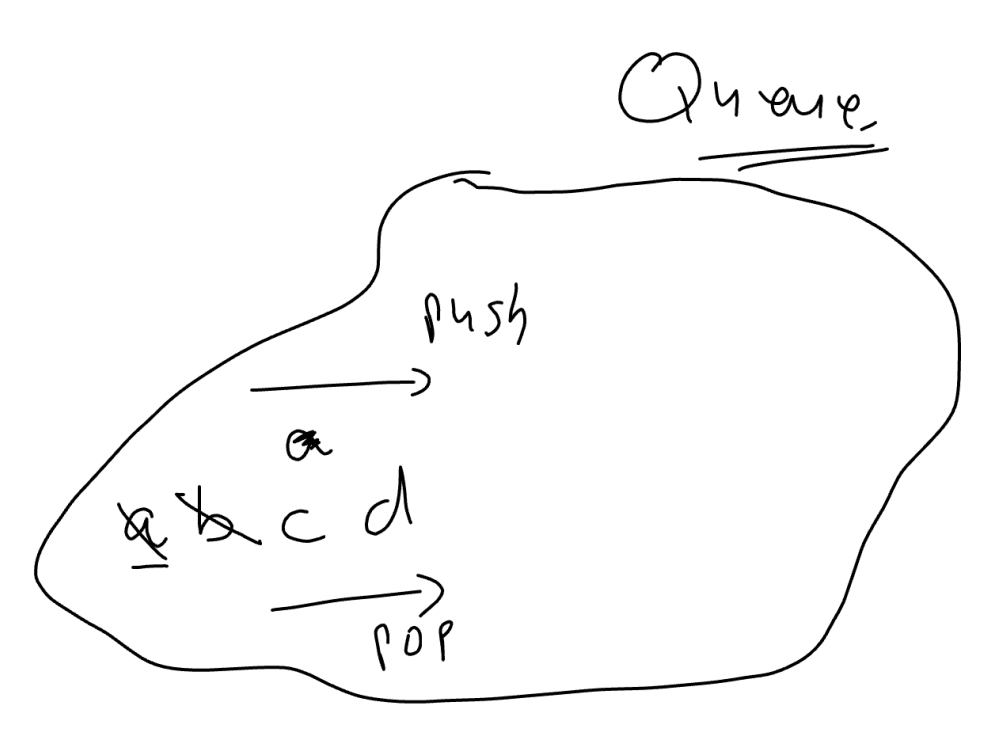
aabc
a # bb

BF: → i: 0 → n-1 [j: 0 → i]

TC: $O(n^2)$
AS: $O(1)$

Queue

a/ø/b/c
a
abacdbd
aabbbcc



a → ~~x~~ 2
b → ~~x~~ 2
c → 1
d → ~~x~~ 2

aabc
(a # bb)

|a|b|c

a → ~~x~~ 2
b → 1
c → 1

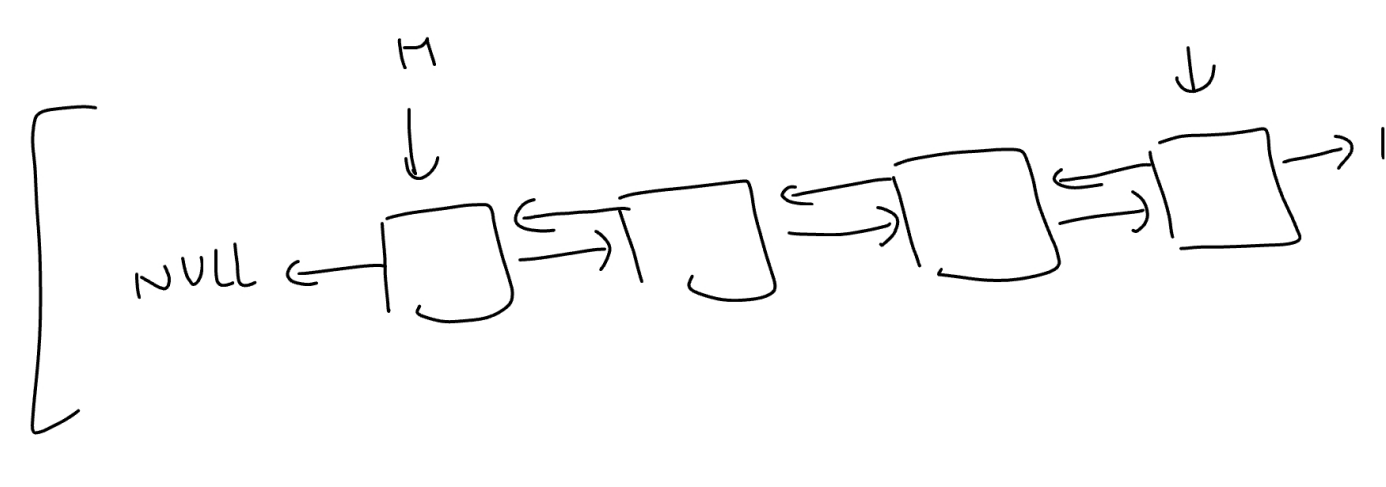
TC: $O(n)$
AS: $O(1)$

[a, b, c - - - - . z]
0 1 2 ... 25

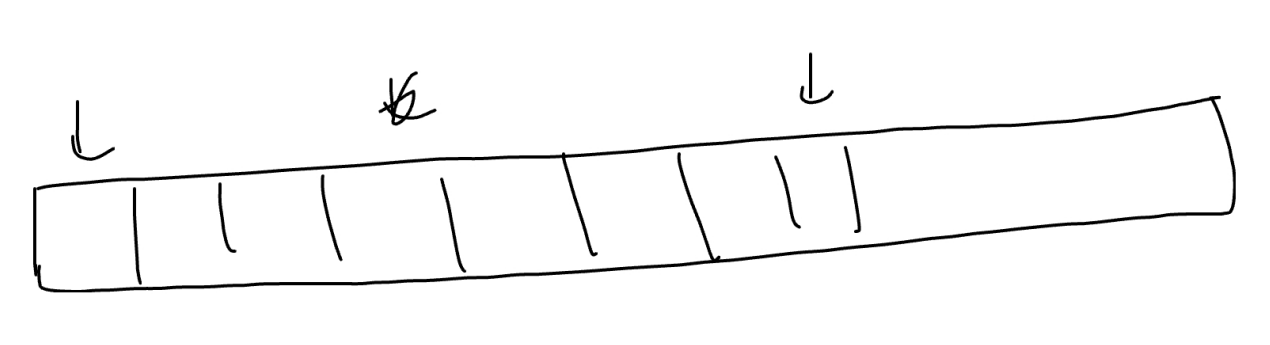
'b' - 'a' = 1

0 → 25

6, 1, 2, 3, 4, 5



Node {
data
* next
* prev
}



Dequeue {
Priv:
:
public: →

}

push_back
pop_back
push_front
pop_front
back()
front()

O(1).

---

0 1 2 3 4 5 6 7 8
[1, 2, 3, 1, 4, 5, 2, 3, 6]

K = 3
n = 9

3 3 4 5 5 5 6

BF: TC: $O((n-k-1) * k) = O(nk - k^2) = O(nk)$
AS: $O(1)$

[1, 2, 3, 1, 4, 5, 2, 10, 10]

3, 3, 4, 5, 5, 5, 2

~~1~~ ~~8~~ ~~3~~, ~~1~~ ~~4~~ ~~5~~, 2, 1
0

monotonically dec.

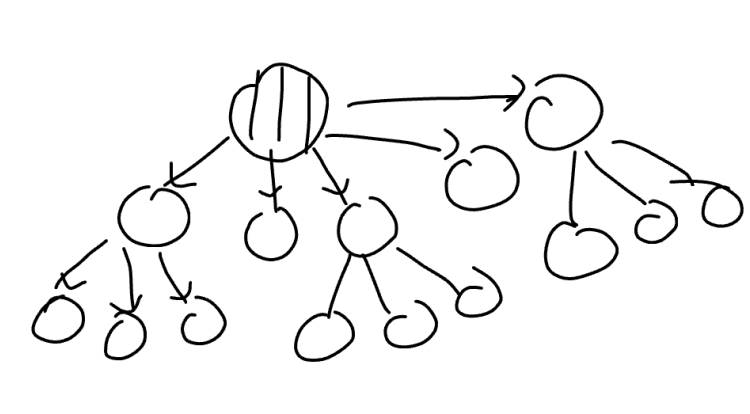
→ front()
push back
pop back
pop front
→ Deque

TC: $O(n)$
AS: $O(K)$

elem = 4

6, 5 3, 1

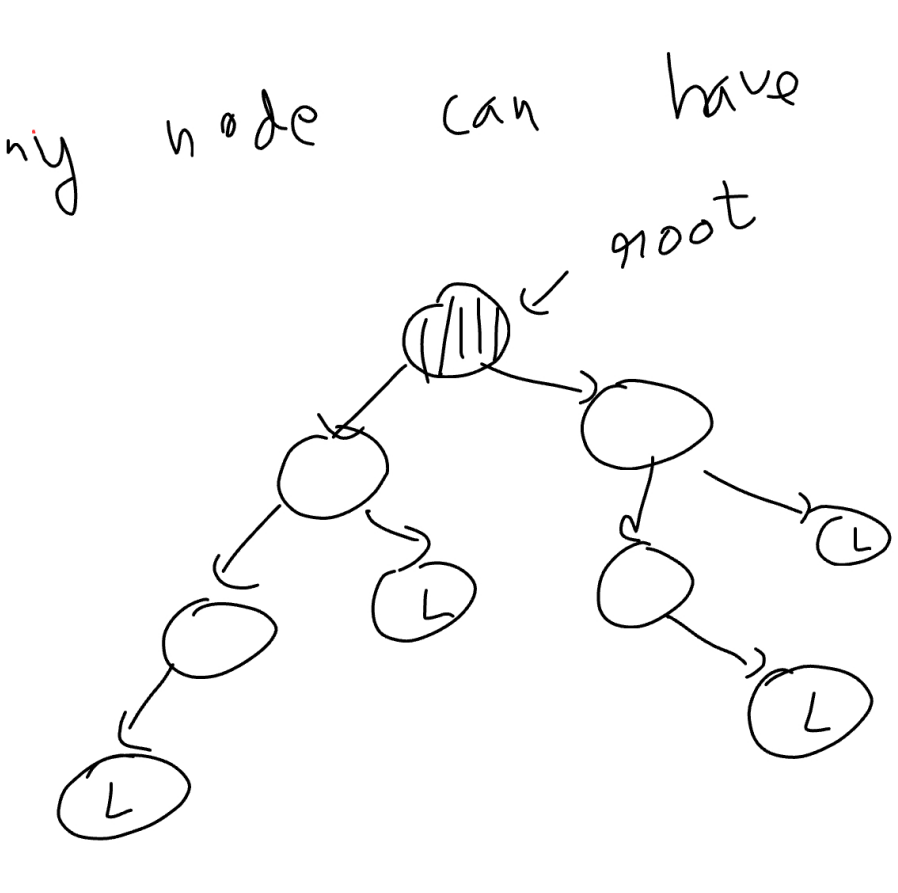
Tree.



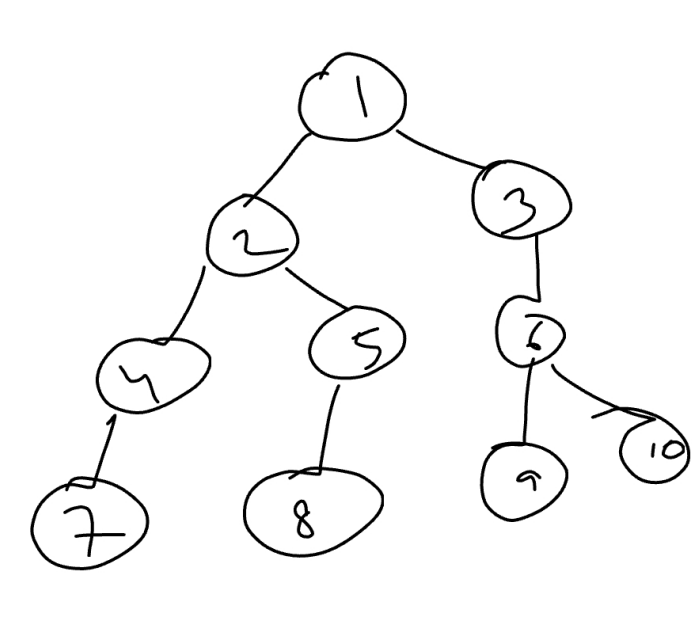
Node {
int data;
vector<Node*> children;
}

Binary Tree

Any node can have at most 2 child nodes.



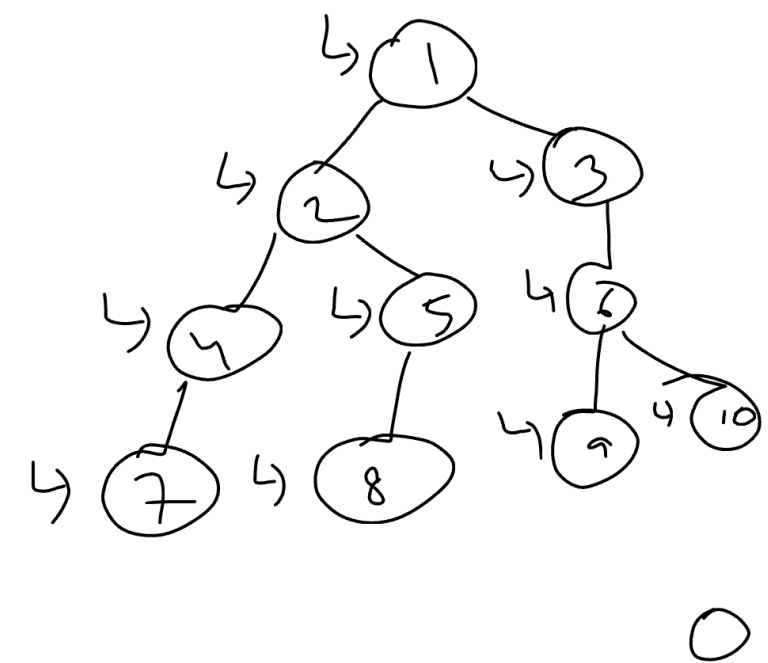
Node {
int data;
Node* left
Node* right
}



Preorder

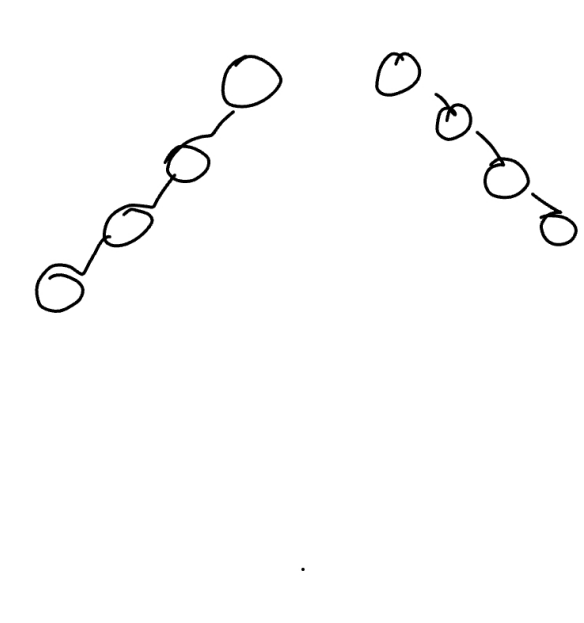
- Self
- Left
- Right

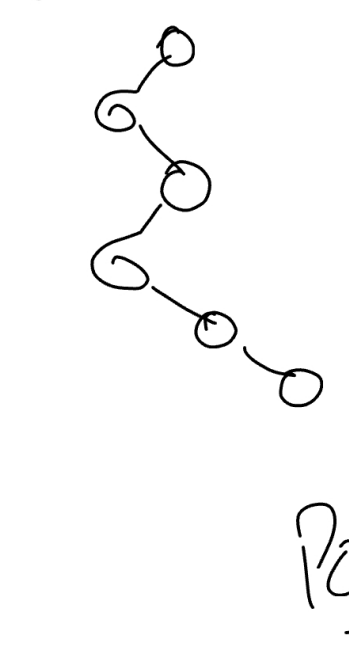
recursive def.



[1, 2, 4, 7, 5, 8, 3, 6, 9, 10]

TC: $O(n)$
AS: $O(H)$



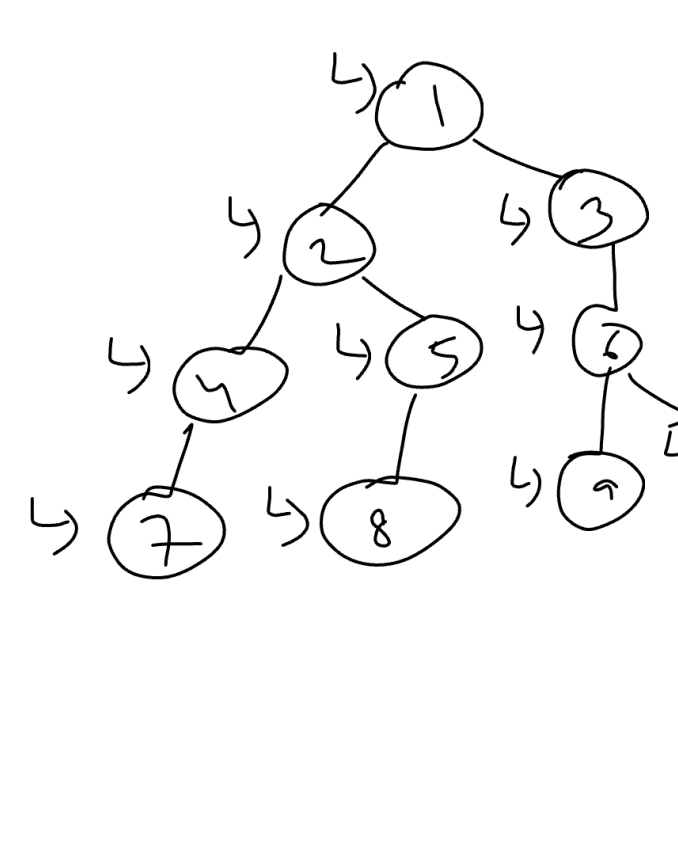


Postorder

→ Left
→ Right
→ Self

rec. def.

TC: $O(n)$
AS: $O(H)$



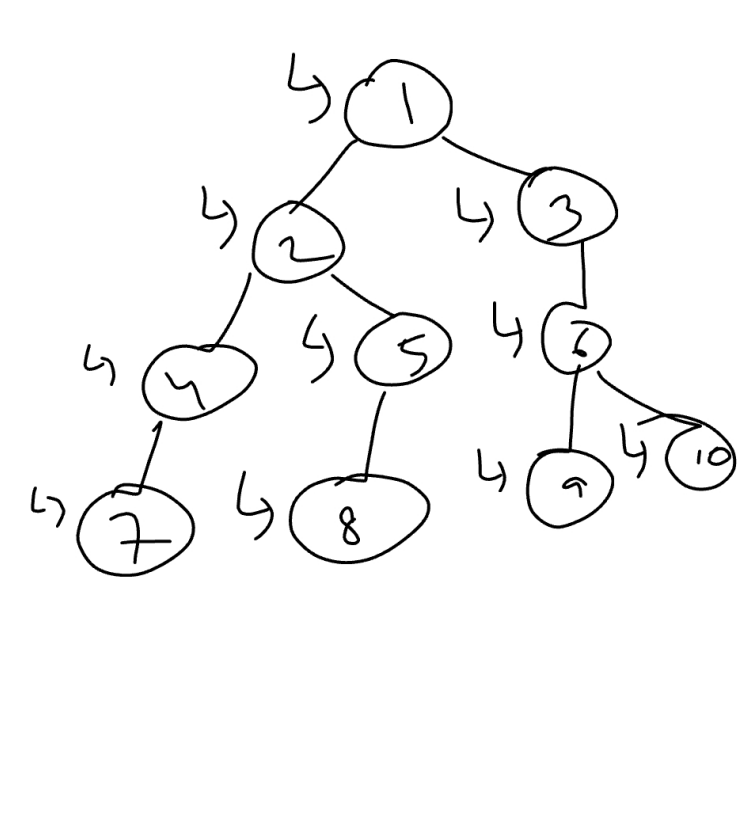
[7, 4, 8, 5, 2, 9, 10, 6, 3, 1]

Inorder

→ Left
→ Self
→ Right

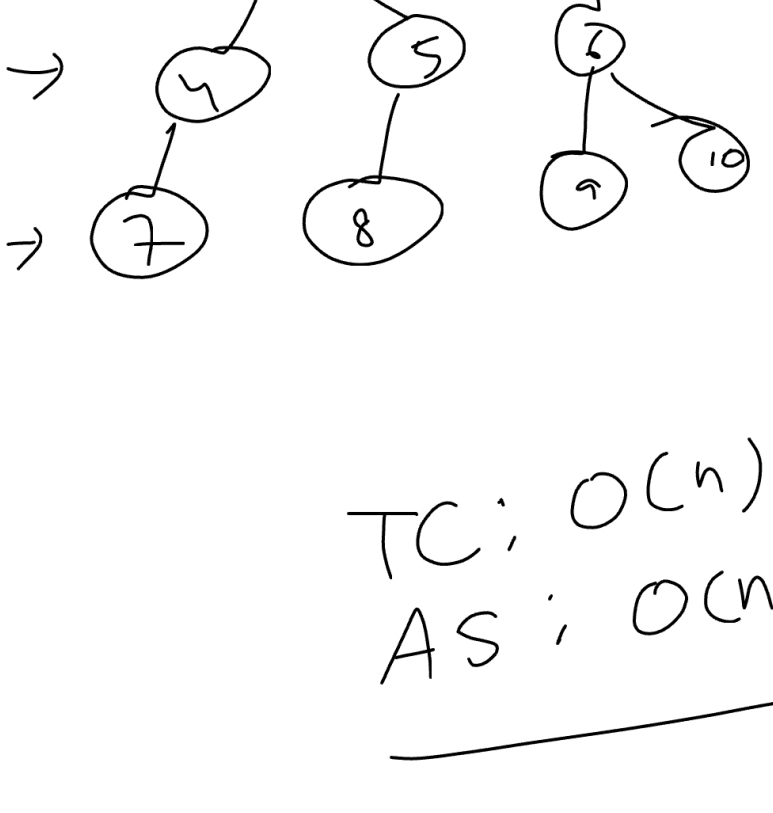
Rec. def.

TC: $O(n)$
AS: $O(H)$.



[7, 4, 2, 8, 5, 1, 9, 6, 10, 3]

Level Order

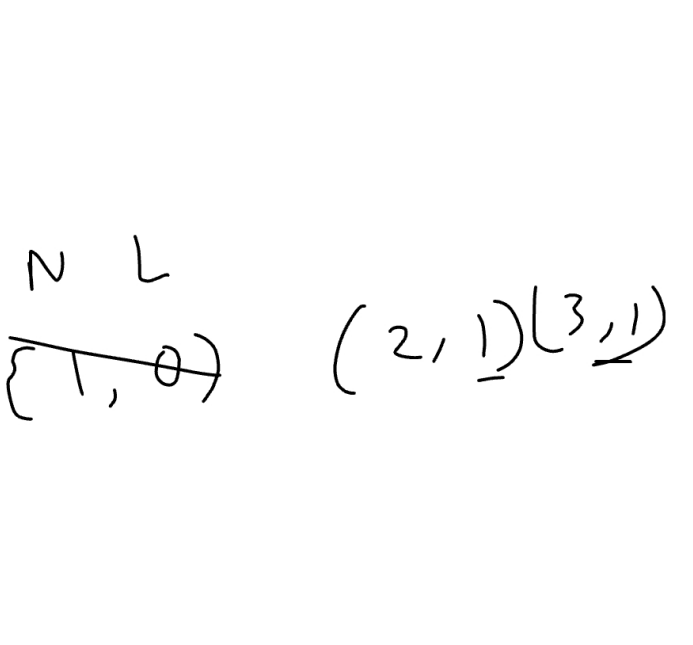


~~1~~ ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~, ~~7~~, 8, ~~9~~, ~~10~~

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

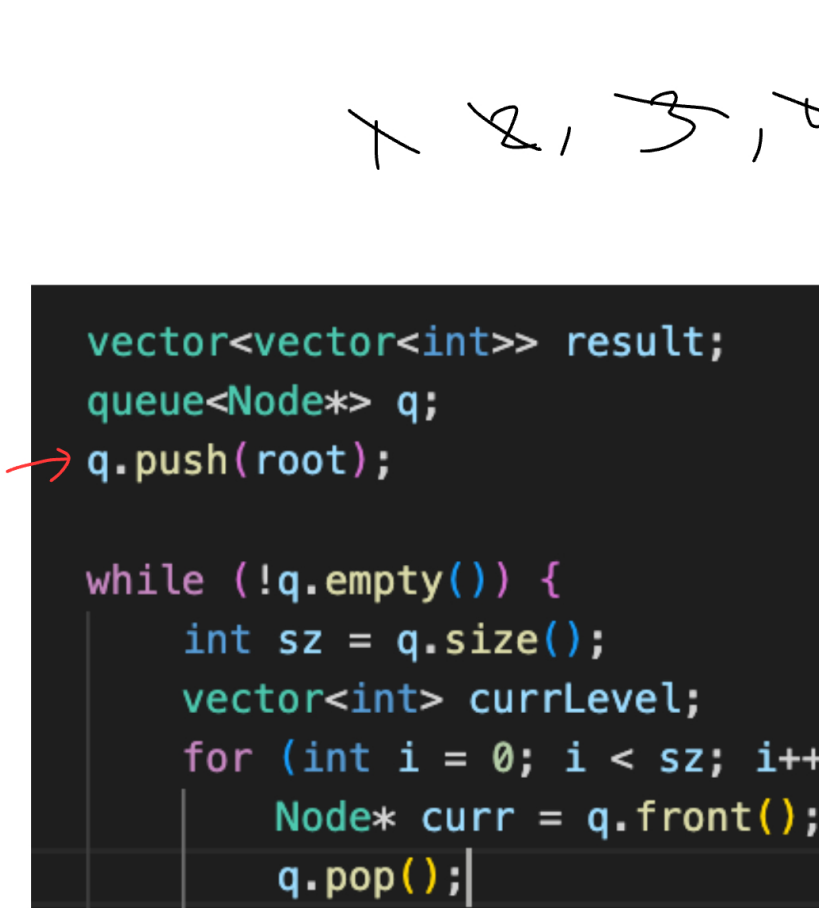
TC: $O(n)$
AS: $O(n)$

$H = \log_2(n)$



$2^{\log_2(n)} = O(n)$

[[1]
[2, 3]
[4, 5, 6]
[7, 8, 9, 10]]



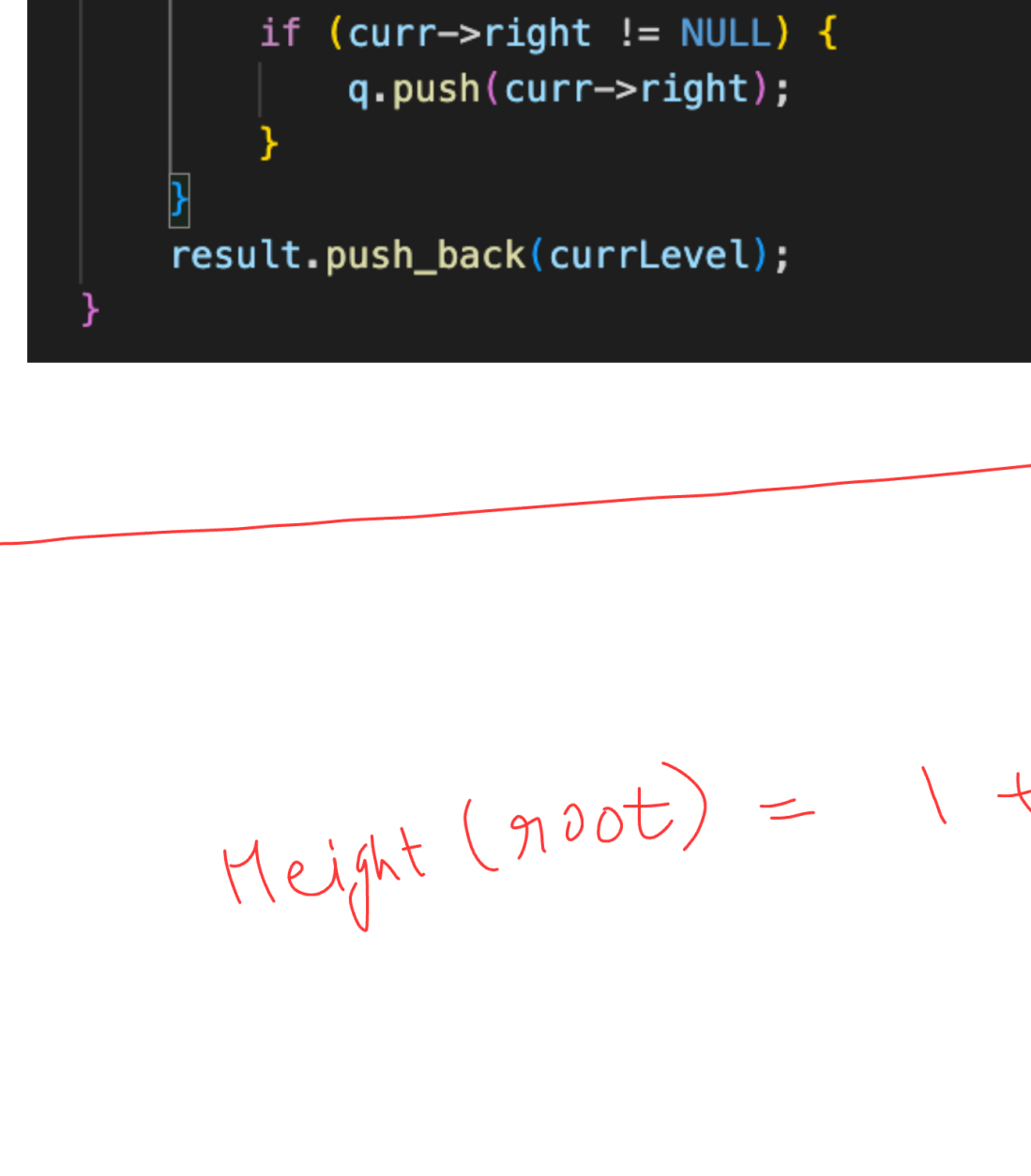
~~1~~ NULL ~~2~~ 3 NULL 4 5 6 NULL

N L
~~(1, 0)~~ (2, 1)(3, 1)

~~1~~ ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~, ~~7~~, ~~8~~, 9, 10

sz = ~~1~~ ~~2~~ ~~3~~ 4

[[1]
[2, 3]
[4, 5, 6]
[7, 8, 9, 10]]



```
vector<vector<int>> result;
queue<Node*> q;
q.push(root);

while (!q.empty()) {
    int sz = q.size();
    vector<int> currLevel;
    for (int i = 0; i < sz; i++) {
        Node* curr = q.front();
        q.pop();

        currLevel.push_back(curr->data);

        if (curr->left != NULL) {
            q.push(curr->left);
        }
        if (curr->right != NULL) {
            q.push(curr->right);
        }
    }
    result.push_back(currLevel);
}
```

~~1~~ ~~2~~, ~~3~~, 4, 5, 6

sz = ~~1~~ ~~2~~ 3

[1]
[2, 3]

Height.

Height(root) = 1 + ~~Height~~ max(Height(root → left), Height(root → right)).